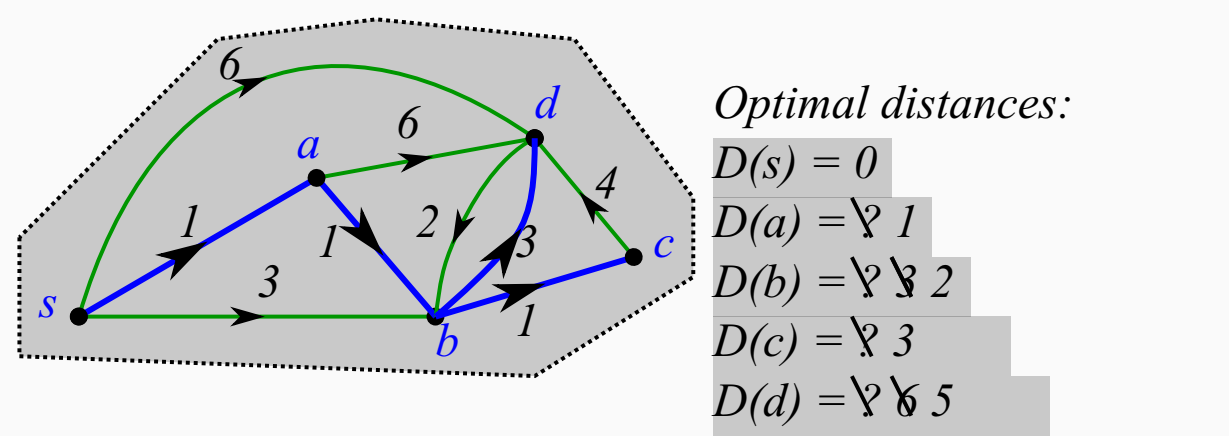
Given:	Digraph G, positive edge weights $w(e)$, and a start vertex s.
Compute:	Shortest (s, u)-paths for every $u \in V(G)$.

Main Idea: Prefixes of shortest paths are shortest paths.



Yes, the blue tree shows shortest (s, u)-paths!

Dijkstra's Algorithm

foreach $v \in V(G)$ **do** $D(v) \leftarrow \infty$, $pred(v) \leftarrow$ NIL
Set $d(s) \leftarrow 0$, Priority Queue $Q \leftarrow V(G)$
while $Q \neq \emptyset$ **do**
    $u \leftarrow$ Extract-Min$(Q, D())$
    foreach *edge* $e = (u, v)$ **do**
        if $D(v) > D(u) + w(e)$ **then**
            $D(v) \leftarrow D(u) + w(e)$ (Decrease-key)
            $pred(v) \leftarrow u$

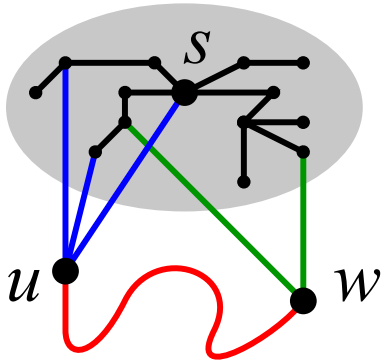
Dijkstra's Algorithm (digraph G, start vertex s): Correctness

- Invariant (1): For each visited node u, $D(u) = |\text{shortest } (s,u)\text{-path}|$, and $pred(u)$ precedes u on such a path.
- Invariant (2): For each unvisited node v, $D(v)$ shortest distance from s to v using only visited nodes, $pred(v)$ precedes v on such a path.
- **Observation**: Prefixes of Shortest-Paths are shortest-paths.

Consider an unvisited node u with smallest $D(u)$.
Claim: A shortest (s,u)-path uses only visited vertices, i.e., $D(v)$ is the length of shortest (s,u)-path. □

foreach $v \in V(G)$ **do** $D(v) \leftarrow \infty$, $pred(v) \leftarrow$ NIL
Set $D(s) \leftarrow 0$, Priority Queue $Q \leftarrow V(G)$
while $Q \neq \emptyset$ **do**
  $u \leftarrow$ Extract-Min$(Q, D())$
  foreach *edge* $e = (u, v)$ **do**
    if $D(v) > D(u) + w(e)$ **then**
      $D(v) \leftarrow D(u) + w(e)$ (Decrease-key)
      $pred(v) \leftarrow u$
return Tree T where $V(T) = V(G)$ and
$E(T) = \{(pred(v), v) : v \in V(G) \setminus \{s\}\}$

Proof of the claim:



$D(w) \geq D(u)$, thus going to w before u not shorter than direct to u.

Dijkstra's Algorithm (digraph G, start vertex s): Running Time

Depends on Extract-Min and Decrease-Key operations. Let $n = |V(G)|, m = |E(G)|$.

- General idea: $O(n \cdot t(\text{Extract-Min}) + m \cdot t(\text{Decrease-key}))$.

Some implementation options:

Data Structure	Extract-Min	Decrease-Key	Total
array/linked-list	$O(n)$	$O(1)$	$O(m + n^2)$
binary-heap	$O(\log n)$	$O(\log n)$	$O((n + m) \log n)$
Fibonacci-heap*	$O(\log n)$	$O(1)$**	$O(m + n \log n)$

*[GT 5.5] [CLRS 19].
**Amortised Time.

foreach $v \in V(G)$ **do** $D(v) \leftarrow \infty$, $pred(v) \leftarrow$ NIL
Set $D(s) \leftarrow 0$, Priority Queue $Q \leftarrow V(G)$
while $Q \neq \emptyset$ **do**
    $u \leftarrow$ Extract-Min$(Q, D())$
    foreach *edge* $e = (u, v)$ **do**
        if $D(v) > D(u) + w(e)$ **then**
            $D(v) \leftarrow D(u) + w(e)$ (Decrease-key)
            $pred(v) \leftarrow u$
return Tree T where $V(T) = V(G)$ and
$E(T) = \{(pred(v), v) : v \in V(G) \setminus \{s\}\}$

Concluding Remarks on Dijkstra's Algorithm

Theorem

Dijkstra's Algorithm can be used to solve the shortest path problem in $O(m + n \log n)$-time, where $m =$ number of edges, $n =$ number of vertices.

Further notes:

- Invented in 1959: publicity event for Dutch railways.
- General Approach to finding shortest-paths.
- Performance depends on the choice of data structure.
- Practical Implementations:
  - use binary-heaps (Fibonacci-heaps – poorer average case performance)
  - local vs. global information: only store the neighbourhood.
  - build shortest path tree only to desired target vertices.